

Curso de Go

Apostila – Concorrência, Paralelismo e Comunicação entre Goroutines

Introdução

Uma das maiores vantagens da linguagem Go é a facilidade de desenvolver programas capazes de executar várias tarefas ao mesmo tempo. Enquanto em outras linguagens trabalhar com múltiplas threads costuma ser complexo, Go foi projetada desde o início para tornar esse tipo de programação simples, segura e eficiente.

Nesta apostila você aprenderá os principais conceitos relacionados à programação concorrente, entenderá a diferença entre concorrência e paralelismo, conhecerá como funciona o escalonador (scheduler) do Go e descobrirá por que goroutines são muito mais leves do que threads tradicionais.

O que é uma Thread?

Uma **thread** é uma unidade de execução criada pelo sistema operacional.

Imagine um programa como uma fábrica. Cada funcionário da fábrica representa uma thread. Enquanto um funcionário monta um produto, outro embala caixas e um terceiro organiza o estoque.

Cada thread possui:

- pilha (stack) própria
- registradores
- contador de instruções
- contexto de execução

Como as threads pertencem ao sistema operacional, criá-las e alternar entre elas possui um custo relativamente alto.

Exemplo:

Um navegador de internet pode possuir:

- uma thread para desenhar a tela;
- outra para baixar imagens;
- outra para executar JavaScript.

O que é Concorrência?

Concorrência significa que um programa consegue lidar com várias tarefas ao mesmo tempo, mesmo que apenas uma esteja realmente sendo executada em determinado instante.

Não significa necessariamente que tudo esteja sendo executado simultaneamente.

O sistema alterna rapidamente entre as tarefas, dando a impressão de que todas acontecem ao mesmo tempo.

Imagine um professor respondendo dúvidas de vários alunos.

Ele responde um aluno por alguns segundos.

Depois responde outro.

Depois outro.

Depois volta ao primeiro.

Todos estão sendo atendidos concorrentemente.

O que é Paralelismo?

Paralelismo ocorre quando duas ou mais tarefas realmente executam ao mesmo tempo.

Isso normalmente acontece quando o computador possui múltiplos núcleos (cores).

Imagine quatro cozinheiros preparando quatro pratos diferentes.

Cada um trabalha simultaneamente.

Esse é o verdadeiro paralelismo.

Concorrência x Paralelismo

Embora muitas pessoas confundam esses conceitos, eles são diferentes.

Concorrência está relacionada à organização das tarefas.

Paralelismo está relacionado à execução simultânea dessas tarefas.

Podemos resumir assim:

Concorrência é lidar com várias tarefas.

Paralelismo é executar várias tarefas ao mesmo tempo.

Uma analogia clássica:

Uma pessoa cozinha arroz, feijão e carne alternando entre as panelas.

Isso é concorrência.

Três cozinheiros preparando um prato cada.

Isso é paralelismo.

O Scheduler do Go

Go possui um componente chamado Scheduler.

Ele é responsável por decidir qual goroutine será executada em cada instante.

O programador não precisa controlar isso manualmente.

O Scheduler distribui automaticamente o trabalho entre os processadores disponíveis.

O modelo M:N

Uma das maiores vantagens do Go é utilizar o modelo M:N.

Mas o que isso significa?

M representa o número de Goroutines.

N representa o número de Threads do sistema operacional.

Em muitas linguagens existe uma relação 1:1.

Cada thread criada pelo programa corresponde a uma thread do sistema operacional.

Go funciona de maneira diferente.

Milhares de goroutines podem compartilhar poucas threads.

Exemplo:

100.000 goroutines

↓

Scheduler

↓

8 Threads

↓

8 núcleos do processador

Essa estratégia reduz drasticamente o consumo de memória e melhora o desempenho.

O que é uma Goroutine?

Uma goroutine é uma função que executa de forma concorrente.

Ela é extremamente leve.

Enquanto uma thread normalmente ocupa vários megabytes de memória, uma goroutine inicia utilizando aproximadamente 2 KB de stack, crescendo conforme necessário.

Criar milhares de goroutines é algo perfeitamente comum em Go.

Comunicação entre Goroutines

Se várias goroutines estão executando ao mesmo tempo, como elas trocam informações?

Go utiliza Channels.

Um channel funciona como um canal de comunicação.

Em vez de compartilhar memória diretamente, as goroutines compartilham mensagens.

Essa abordagem reduz erros e torna o código muito mais seguro.

O Modelo Produtor-Consumidor

Um dos padrões mais importantes da programação concorrente é o modelo Produtor-Consumidor.

Nesse modelo existem dois papéis.

Produtor

É responsável por gerar tarefas.

Pode ser:

- usuários acessando um site;

- sensores IoT;
- pedidos de uma loja virtual.

O produtor apenas envia informações.

Consumidor

É responsável por processar essas informações.

Pode ser:

- servidor web;
- banco de dados;
- worker;
- sistema de pagamentos.

Os consumidores recebem tarefas através dos channels.

Imagine um restaurante.

Garçom

↓

Cozinha

O garçom recebe pedidos.

A cozinha prepara os pratos.

O garçom é o produtor.

A cozinha é o consumidor.

Workers

Um Worker é um consumidor especializado.

Em vez de existir apenas um consumidor, podemos criar vários workers.

Exemplo:

Worker 1

Worker 2

Worker 3

Todos retiram tarefas da mesma fila.

Isso aumenta muito o desempenho.

Buffered Channels

Um channel comum funciona como um telefone.

Quem envia precisa esperar alguém atender.

Já um Buffered Channel funciona como uma caixa de correio.

É possível colocar algumas cartas lá dentro antes que alguém venha buscá-las.

Isso reduz bloqueios temporários.

Multiplexação

Imagine um sistema aguardando respostas de:

um banco de dados

uma API

um servidor

Sem o select seria necessário verificar cada canal manualmente.

O select faz isso automaticamente.

Ele espera vários channels simultaneamente e executa aquele que responder primeiro.

Timeout

Nem sempre uma tarefa termina.

Às vezes:

o servidor cai;

uma API demora;

o banco trava.

Esperar para sempre seria um erro.

Por isso utilizamos timeout.

Após determinado tempo, o programa desiste da operação e continua executando normalmente.

O que é Deadlock?

Deadlock acontece quando todas as goroutines ficam esperando umas pelas outras.

Nenhuma consegue continuar.

Exemplo:

Uma goroutine tenta enviar um valor para um channel.

Mas ninguém está lendo.

Ela fica bloqueada.

Se todas fizerem isso, o programa trava completamente.

Go normalmente exhibe:

fatal error: all goroutines are asleep - deadlock!

O que é Race Condition?

Race Condition ocorre quando duas ou mais goroutines modificam a mesma variável ao mesmo tempo.

O resultado passa a depender da ordem em que elas executam.

Isso gera resultados imprevisíveis.

Exemplo:

Saldo = R\$100

Goroutine A soma 50.

Goroutine B subtrai 20.

Dependendo da ordem de execução, o resultado pode variar.

Go oferece ferramentas como Mutex e Channels para evitar esse problema.

O que é Starvation?

Starvation significa "inanição".

Uma goroutine nunca consegue executar porque outras estão utilizando constantemente os recursos disponíveis.

Imagine uma fila de supermercado onde pessoas VIP sempre passam na frente.

Quem está no final da fila talvez nunca seja atendido.

Vazamento de Goroutines

Outro problema comum é criar goroutines que nunca terminam.

Elas ficam:

esperando dados;

presas em channels;

aguardando eventos.

Com o tempo, milhares dessas goroutines acumuladas desperdiçam memória e processamento.

Esse problema é conhecido como Goroutine Leak.

Boas Práticas

Ao trabalhar com concorrência em Go procure sempre:

- utilizar channels para comunicação;
- evitar compartilhar variáveis entre goroutines;
- fechar channels quando não forem mais utilizados;
- usar buffered channels quando houver filas;
- utilizar select para múltiplos eventos;
- definir timeouts em operações externas;
- evitar criar goroutines sem necessidade.